

 Unified_Engine...
281.32 KB

Act as an expert Quant Developer and Chromium Automation Engineer. I need you to perform a line-by-line code audit, find potential bugs, identify race conditions, and recommend optimizations for the attached Python file ('Unified_Engine_3.py').

```
### Context & Stack:
- **Project** -- Real-time crypto trading engine.
- **Stack** -- Playwright (CDP connection to existing Chrome), Binance API & WebSocket (obd/trades), LightGBM/CatBoost (ML live inference), Rich (Terminal TUI), and an HTTP Web Dashboard.
- **Data flow** -- 18 assets. Layout scraper (TV layout 's9' across 2 tabs) extracts metrics (Premium Index, CVDs, OI, Funding, L/S Ratio, Liquidations) and updates a central 'UnifiedDataFrame'. A background thread feeds trades to a 'FootprintScraper' to build real-time candle order flow profiles.
```

1. **Thread-Safety & Race Conditions:** Check the locking mechanisms around 'UnifiedDataFeed' and 'footprint_store' (e.g., 'update_from_layout', 'update_from_footprint', and strategy evaluations). Are there any potential deadlocks or race conditions?
2. **CDP & Playwright Robustness:** Analyze how browser context pages are fetched, sorted, navigated, and evaluated in 'get_sorted_pages' and 'get_live_data()' . Can any network/navigation errors hang the background threads indefinitely?
3. **Data Integrity & Failbacks:** Review how layout values (often suffixed with 'K', 'M', etc.) are parsed to floats. Ensure the fallback mechanisms (like Binance footprint' → stock failing vs layout price failing back) do not overwrite active/live data with stale feed parameters.
4. **Memory Management:** Ensure historical data arrays ('self.history') are pruned efficiently and don't leak memory over weeks of continuous execution.
5. **Efficiency & Performance:** Suggest optimizations for reducing CPU usage in the Rich live TUI update loop and the Playwright evaluation loop ('EXTRACTION_JS').

Please format your response into a table of identified risks/bugs, followed by specific diff-style code corrections for any high-priority issues found.

Below is a functional line-by-line/block audit. I'm anchoring findings by class/function because the pasted monolith has no stable line numbers.

High-risk findings

Severity	Area	Risk / bug	Why it matters	Recommended fix
Critical	meta_labeler.py _API_KEYS	Hardcoded Google/Gemini API keys are embedded in the script.	Keys are now exposed and should be considered compromised.	Remove from code, load from env vars, rotate/revoke leaked keys.
High	TradeTracker	active_trade_history, current_capital, peak_equity are mutated from the strategy eval thread and the TUU/main thread without a lock.	Race can double-close, miss closes, corrupt capital, or render partially-mutated trades.	Add threading.Lock, lock process_signal, update_live_peak, get_stats; render from snapshots.
High	AlphaSqueezerV17_compute_features	Directly reads self.feed.history without UnifiedDataFeed.lock.	Concurrent append/prune can produce inconsistent Coinglass features.	Add UnifiedDataFeed.get_symbol_history_snapshot() and use it.
High	AlphaSqueezerV17.__init__/_config	Strategy name is AlphaSqueezer_V17_BTC, etc., but STRATEGY_CONFIG key is AlphaSqueezer_V17.	Per-asset config overrides and default V17 cooldown are silently ignored.	Add CONFIG_NAME = "AlphaSqueezer_v17" and override get_param() to use it.
High	Strategy eval worker	last_signal_time is never updated after a signal. Cooldown check is therefore ineffective.	Strategy can immediately re-trigger after exit, active trades block duplicates but API/Gemini/ML calls still spam.	Have process_signal() return accepted boolean; set last_signal_time on accepted entries; skip active trades.
High	AlphaSqueezerV17.evaluate_trigger	_feature_cache exists but is unused; the worker recomputes 1000-row DataFrames for every symbol every ~0.1s.	Major CPU burn and unnecessary Binance/cache churn.	Cache feature DataFrame per latest kline open time with short TTL, and evaluate each strategy every ~5s, not 10x/sec.
High	UnifiedDataFeed.update_frow_footprint/price	Footprint price only updates feed price if current price is exactly 0.0; stale layout price can dominate forever.	Trades may use stale browser layout price while Binance websocket has fresh price.	Track price source/timestamp ; prefer fresh footprint price when available; layout only as fallback.
High	get_sorted_pages() during live scraping	Live get_live_data() calls get_sorted_pages(), which can navigate/open tabs as a side effect.	Background scraper can mutate tabs, cause layout resets, or block on navigation during trading.	Split discovery/setup from steady-state scraping, live mode should use cached pages and avoid navigation unless recovery is explicit.
		page.evaluate()		Run Playwright scraper in a...

High	Playwright sync CDP calls	77 CDP calls can block a Python thread indefinitely if the renderer/CDP target hangs.	Background TV scraper can silently die/hang with no recovery.	separate process with heartbeat, or at minimum set timeouts, cache pages, reconnect on failure.
High	Main loop	Writes <code>main_loop_debug.log</code> twice every 250ms.	Disk bloat and avoidable I/O latency over long runtimes.	Remove or gate behind debug flag with rotation/throttling.
Medium/High	FootprintScraper / TARGET_ASSETS	XAUUSD, XAGUSD, CLUSD, NATGASUSD may not be valid Binance USD-M futures symbols. 	REST bootstrap and websocket combined streams may error/reconnect repeatedly; raw kline strategy calls fail.	Filter Binance-backed symbols via <code>exchangeInfo</code> ; keep layout-only assets separate.
Medium	BaseTrendStrategy.get_raw_klines	Cache key omits limit.	A prior <code>limit=100</code> call can poison a later <code>limit=1000</code> V17 call.	Use cache key (<code>symbol</code> , <code>interval</code> , <code>limit</code>).
Medium	<code>parse_value_to_float</code>	Parser does not robustly handle unicode minus, T suffix, lowercase suffixes, mixed tokens, malformed values.	Coinglass values can parse to <code>0.0</code> incorrectly, causing false signals/anomalies.	Harden parser.
Medium	UnifiedDataFeed.add_to_history	Uses repeated <code>pop(0)</code> for pruning.	$O(n)$ per pop; over weeks this is unnecessary CPU.	Batch delete old prefix or use deque.
Medium	TradeLogger._write_to_disk	Loads/rewrites whole JSON log on every entry/exit.	Gets slower as logs grow, risk during crashes.	Use JSONL append log + compacted summary periodically
Medium	<code>browser.close()</code> after CDP setup	Closing a CDP connected browser can terminate/disconnect the browser unexpectedly depending on Playwright/CDP behavior. 	Background scraper may fail if Chrome is closed.	Keep setup CDP connection open, or explicitly manage lifecycle at shutdown only
Medium	FootprintScraper.stop	Calls <code>loop.stop()</code> while <code>while _listen()</code> future may still be running; thread is not joined.	Runtime warnings/leaked thread on shutdown.	Signal stop, close websocket, cancel tasks, join thread.
Medium	EXTRACTION_JS	Uses <code>innerText</code> and scans every legend in every iframe every scrape.	<code>innerText</code> can trigger layout; repeated DOM scans are expensive.	Use <code>textContent</code> where possible; cache selectors/function in page, scrape only known legend nodes.
Medium	<code>strategy_eval_worker</code>	Catches exceptions and sleeps with logging stack traces. 	Strategy thread can repeatedly fail with no visibility.	Log exceptions to file/stdout with <code>traceback.format_exc()</code> .
Medium	Path resolution	Several classes use <code>os.path.dirname(os.path.dirname(os.path.abspath(__file__)))</code> ; in a monolithic script this points outside the project.	Logs/models may be written/read from wrong directories.	Pass explicit paths or define one <code>PROJECT_DIR = os.path.dirname(__file__)</code> .
Medium	HAS_LGB guard	<code>import lightgbm as lgb</code> happens before the <code>try</code> , so missing LightGBM crashes before fallback.	"Passthrough mode" never activates.	Wrap import itself in <code>try/except</code> .

High-priority code corrections

1. Remove hardcoded API keys and fix LightGBM import guard

```
##
-import lightgbm as lgb
+try:
+    import lightgbm as lgb
+    HAS_LGB = True
+except ImportError:
+    lgb = None
+    HAS_LGB = False
```

Remove the later ineffective block:

```
## Try importing LightGBM (required for predictions)
+try:
+    HAS_LGB = True
+except ImportError:
+    HAS_LGB = False
```

Replace the hardcoded Gemini key list:

```
## API keys = f
```

```

_API_KEYS = [
    "... hardcoded keys ..."
]

if not _API_KEYS and os.environ.get("GEMINI_API_KEY"):
    _API_KEYS.append(os.environ.get("GEMINI_API_KEY"))
_API_KEYS = [
    k.strip()
    for k in os.environ.get("GEMINI_API_KEYS", "").split(",")
    if k.strip()
]
if os.environ.get("GEMINI_API_KEY"):
    _API_KEYS.append(os.environ["GEMINI_API_KEY"].strip())

```

Also rotate/revoke the exposed keys.

2. Harden parse_value_to_float

```

def parse_value_to_float(val):
    if not val or val == "N/A" or val == "0" or val.startswith("Loading"):
        if val is None:
            return 0.0
        is_pct = "%" in val
        val = val.replace(" ", "").replace(",", "").replace("$", "").replace("%", "")
        val = str(val).strip()
        if (
            not val
            or val.upper() in ["N/A", "NA", "NONE", "NULL", "NAN", "--"]
            or val.startswith("Loading")
        ):
            return 0.0
        # Normalize common Coinglass/TV glyphs.
        val = (
            val.replace("0", "0")
            .replace("-", "-")
            .replace("\u2212", "-")
            .replace(",", "")
            .replace("$", "")
            .strip()
        )
        # If a legend accidentally returns "1.2M 3.4M", parse first token.
        if " " in val:
            val = val.split()[0]
        is_pct = "%" in val
        val = val.replace("%", "")
        multiplier = 1.0
        if val.upper().endswith("K"):
            suffix = val[-1:].upper()
            if suffix == "K":
                multiplier = 1000.0
                val = val[:-1]
            elif val.upper().endswith("M"):
                multiplier = 1000000.0
                val = val[:-1]
            elif val.upper().endswith("B"):
                multiplier = 1000000000.0
                val = val[:-1]
            elif suffix == "T":
                multiplier = 1000000000000.0
                val = val[:-1]
        try:
            parsed_val = float(val) * multiplier
            if is_pct:
                return parsed_val / 100.0
            return parsed_val
        except:
            except Exception:
                return 0.0

```

3. Fix UnifiedDataFeed locking, pruning, and stale price priority

```

class UnifiedDataFeed:
    def __init__(self):
        self.latest_data = collections.defaultdict(dict)
        self.history = collections.defaultdict(lambda: collections.defaultdict(list))
        self.lock = threading.Lock()
        self.history_retention_seconds = 120 * 3600
        self.history_bucket_seconds = 30
        self.price_source_ttl_seconds = 15.0
        def get_symbol_history_snapshot(self, symbol, keys=None):
            """
            Lock-safe copy of one symbol's history.
            Prevents strategy feature engineering from iterating lists while writers
            prune/append.
            """
            with self.lock:
                src = self.history.get(symbol, {})
                if keys is None:
                    return (k: list(v) for k, v in src.items())
                return {k: list(src.get(k, [])) for k in keys}
    def add_to_history(self, symbol, key, val):
        BUCKET_SECONDS = 30
        BUCKET_SECONDS = self.history_bucket_seconds
        bucket_t = int(time.time() / BUCKET_SECONDS) * BUCKET_SECONDS
        hist = self.history[symbol][key]
        if not hist or hist[-1][0] < bucket_t:
            hist.append((bucket_t, val))
        else:
            hist[-1] = (bucket_t, val)
        # trim history to last 120 hours (lookback-friendly) using O(1) memory footprint
        popping
        current_time = time.time()
        while hist and current_time - hist[0][0] > 432000:
            hist.pop(0)
        cutoff = current_time - self.history_retention_seconds
        # Batch prune instead of repeated pop(0).
        if hist and hist[0][0] < cutoff:
            cut = 0
            for cut, (ts, _) in enumerate(hist):

```

```

        if ts >= cutoff:
            break
        else:
            cut = len(hist)
            if cut:
                del hist[:cut]

        max_points = int(self.history_retention_seconds / self.history_bucket_seconds) + 10
        if len(hist) > max_points:
            del hist[:max_points]

    @@
    def update_from_layout(self, layout_data):
        with self.lock:
            now = time.time()
            for symbol, data in layout_data.items():
                for k, v in data.items():
                    if k.endswith('_history'):
                        continue
                    # if k == 'price':
                    #     # Ignore layout scraper price; we use footprint API price as
                    #     # single source of truth
                    #     continue

                    if k == 'price':
                        try:
                            price_val = float(v)
                        except Exception:
                            price_val = parse_value_to_float(v)

                        if price_val <= 0:
                            continue

                        # Do not overwrite a fresh Binance/footprint price with browser-
                        # layout price.
                        src = self.latest_data[symbol].get('_price_source')
                        ts = float(self.latest_data[symbol].get('_price_ts', 0.0) or 0.0)
                        if src == 'footprint' and (now - ts) <= self.price_source_ttl_seconds:
                            continue

                        self.latest_data[symbol]['price'] = price_val
                        self.latest_data[symbol]['_price_source'] = 'layout'
                        self.latest_data[symbol]['_price_ts'] = now
                        self.add_to_history(symbol, 'price', price_val)
                        continue

                    self.latest_data[symbol][k] = v
                    self.add_to_history(symbol, k, v)
                    if k == 'ls_ratio':
                        self.latest_data[symbol]['macro_long_short_ratio'] = v

    @@
    def update_from_footprint(self):
        with self.lock:
            with footprint_store_lock:
                for symbol, data in footprint_store.items():
                    price = data.get('price', 0.0)
                    if price > 0.0 and self.latest_data.get(symbol, {}).get('price', 0.0)
                    == 0.0:
                        self.latest_data[symbol]['price'] = price
                        self.add_to_history(symbol, 'price', price)

                # Avoid nested feed.lock -> footprint_store_lock.
                with footprint_store_lock:
                    fp_snapshot = {s: d.copy() for s, d in footprint_store.items()}

                now = time.time()
                with self.lock:
                    for symbol, data in fp_snapshot.items():
                        price = float(data.get('price', 0.0) or 0.0)
                        last_update = float(data.get('last_update', 0.0) or 0.0)
                        fresh = (now - last_update) < self.price_source_ttl_seconds

                        if price > 0.0 and fresh:
                            self.latest_data[symbol]['price'] = price
                            self.latest_data[symbol]['_price_source'] = 'footprint'
                            self.latest_data[symbol]['_price_ts'] = now
                            self.add_to_history(symbol, 'price', price)

                        self.latest_data[symbol]['delta'] = data.get('delta', 0.0)
                        self.latest_data[symbol]['bid_vol'] = data.get('bid_vol', 0.0)
                        self.latest_data[symbol]['ask_vol'] = data.get('ask_vol', 0.0)
                        self.latest_data[symbol]['poc'] = data.get('poc', 'N/A')
                        # delta_percent is used by LCMS ambush confirmation - must be mapped
                        here
                        self.latest_data[symbol]['delta_percent'] = data.get('delta_percent',
                        0.0)
                        self.latest_data[symbol]['of_absorption_bot'] =
                        data.get('of_absorption_bot', 1.0)
                        self.latest_data[symbol]['of_absorption_top'] =
                        data.get('of_absorption_top', 1.0)
                        self.latest_data[symbol]['of_delta_ratio'] = data.get('delta_percent',
                        0.0) / 100.0

                        delta = float(data.get('delta', 0.0) or 0.0)
                        delta_pct = float(data.get('delta_percent', 0.0) or 0.0)

                        self.latest_data[symbol]['delta'] = delta
                        self.latest_data[symbol]['bid_vol'] = data.get('bid_vol', 0.0)
                        self.latest_data[symbol]['ask_vol'] = data.get('ask_vol', 0.0)
                        self.latest_data[symbol]['poc'] = data.get('poc', 'N/A')
                        self.latest_data[symbol]['delta_percent'] = delta_pct
                        self.latest_data[symbol]['of_absorption_bot'] =
                        data.get('of_absorption_bot', 1.0)
                        self.latest_data[symbol]['of_absorption_top'] =
                        data.get('of_absorption_top', 1.0)
                        self.latest_data[symbol]['of_delta_ratio'] = delta_pct / 100.0
                        self.add_to_history(symbol, 'delta', delta)
                        self.add_to_history(symbol, 'delta_percent', delta_pct)

```

Copy

4. Fix raw kline cache key

```

@@ class BaseTrendStrategy
-     key = f"{symbol}_{interval_mins}"
+     key = f"{symbol}_{interval_mins}_{limit}"

```

Copy

5. Fix V17 config lookup and use locked history snapshots

```

class AlphaSqueezerV17(BaseTrendStrategy):
+     CONFIG_NAME = "AlphaSqueezer_V17"

    @@
    def __init__(self, feed, symbol: str, model_dir: str = None):
-         super().__init__(feed, f"AlphaSqueezer_V17_{symbol[:3]}")
+         super().__init__(feed, f"AlphaSqueezer_V17_{symbol[:3]}")

```

Copy

```

+ base.symbol = symbol.replace("USD", "")
+ super().__init__(feed, f"[self.CONFIG_NAME]_{base.symbol}")
+ self.symbol = symbol
@@
+ if os.path.exists(wfo_path):
+     try:
+         with open(wfo_path, 'r') as f:
+             all_params = json.load(f)
+             self.wfo_params = all_params.get(self.symbol, {})
+     except Exception as e:
+         print(f"[self.strategy_name] Error loading WFO params: {e}")
+
+ def get_param(self, symbol: str, param_name: str, fallback=None):
+     # Strategy instance names are unique per asset, but config is stored under base
+     # key.
+     return get_param(symbol, self.CONFIG_NAME, param_name, fallback)
@@
- hist_data = self.feed.history.get(self.symbol, {})
+ cg_keys = [
+     'oi',
+     'ls_ratio',
+     'funding',
+     'liquidations_long_15m',
+     'liquidations_short_15m',
+ ]
+ if hasattr(self.feed, "get_symbol_history_snapshot"):
+     hist_data = self.feed.get_symbol_history_snapshot(self.symbol, cg_keys)
+ else:
+     hist_data = {}

```

Copy

Add actual feature caching in evaluate_trigger():

```

@@
- # Compute features
- df, feat_cols = self._compute_features(klines)
+ # Compute features. Cache briefly because the eval loop runs much faster
+ # than the 15m bar and CoinGlass refresh cadence.
+ latest_open_time = int(klines[-1][0])
+ now_ts = time.time()
+ cached = self._feature_cache
+ if (
+     cached
+     and cached.get("open_time") == latest_open_time
+     and now_ts - cached.get("ts", 0.0) <= 5.0
+ ):
+     df = cached["df"]
+     feat_cols = cached["feat_cols"]
+ else:
+     df, feat_cols = self._compute_features(klines)
+     if df is not None:
+         self._feature_cache = {
+             "open_time": latest_open_time,
+             "ts": now_ts,
+             "df": df,
+             "feat_cols": feat_cols,
+         }
+     if df is None:
+         return None, "Feature computation failed"

```

Copy

6. Make TradeTracker thread-safe and render from snapshots

In TradeTracker.__init__:

```

class TradeTracker:
+ def __init__(self, log_file="trade_logs.json", initial_capital=INITIAL_CAPITAL):
@@
+     self.current_capital = initial_capital
+     self.lock = threading.RLock()
+     self.active_trades = {} # key: (symbol, strat_name), val: dict
+     self.history = []
+     self._load_history()
- self.logger = TradeLogger()
+ self.logger = TradeLogger(log_dir=logs_dir)

```

Copy

Wrap process_signal():

```

+ def process_signal(self, symbol, strat_name, res, current_price):
+     with self.lock:
+         return self._process_signal_locked(symbol, strat_name, res, current_price)
+
+ def _process_signal_locked(self, symbol, strat_name, res, current_price):
+     if not res:
-         return
+         return False

```

Copy

Then inside the moved body:

```

@@ close branch, after self.logger.log_exit(...)
+ self.logger.log_exit(
+     trade_id, exit_price, exit_time, exit_reason, pnl_usd, pnl_pct,
+     duration_seconds, max_pnl, max_loss, exit_context, system_data
+ )
+ return True
@@ entry branch guard returns
- return
+ return False
@@ after self.logger.log_entry(...)
+ self.logger.log_entry(
+     trade_id, symbol, strat_name, direction, entry_price, trade["entry_time"],
+     units, sl, tp, context_data, system_data
+ )
+ return True
+
+ return False

```

Copy

Mechanically replace the other bare guard return statements in the entry branch with return False.

Wrap live PnL updates:

```

+ def update_live_pnl(self, feed):
+     with self.lock:
+         return self._update_live_pnl_locked(feed)
+
+ def _update_live_pnl_locked(self, feed):

```

Copy

```

self.feed = feed
latest = feed.get_latest()

```

Add a safe render snapshot:

```

+ def get_render_snapshot(self):
+     with self.lock:
+         active = {k: v.copy() for k, v in self.active_trades.items()}
+         hist = [t.copy() for t in self.history]
+         stats = self.get_stats()
+         return active, hist, stats

```

In the TUI render section:

```

- stats = trade_tracker.get_stats()
+ active_snapshot, history_snapshot, stats =
trade_tracker.get_render_snapshot()
    trade_table = Table(show_header=True, header_style="bold
bright_magenta", border_style="magenta", expand=True)
- trade_table.add_column(f"Active Trades
((len(trade_tracker.active_trades)))", justify="left")
+ trade_table.add_column(f"Active Trades ({len(active_snapshot)})",
justify="left")
@@
- for k, t in trade_tracker.active_trades.items():
+ for k, t in active_snapshot.items():
@@
- for t in trade_tracker.history[-3:]:
+ for t in history_snapshot[-3:]:

```

7. Fix strategy eval spam, cooldown, and silent exception swallowing

```

def strategy_eval_worker():
    global _SHUTDOWN_REQUESTED
    next_eval_time = collections.defaultdict(float)
    while not _SHUTDOWN_REQUESTED:
        try:
            now = time.time()
            for symbol in TARGET_ASSETS:
                for strat in strategies:
                    strat_locked_symbol = getattr(strat, 'symbol', None)
                    if strat_locked_symbol is not None and strat_locked_symbol !=
symbol:
                        continue
@@
- strat_cooldown = strat.get_param(symbol, 'cooldown_seconds',
1800)
- if "TRIGGERED" in strat.state.get(symbol, "") and \
- time.time() - strat.last_signal_time.get(symbol, 0) >=
strat_cooldown:
+ active_key = (symbol, strat.strategy_name)
+ with trade_tracker.lock:
+     if active_key in trade_tracker.active_trades:
+         continue
+
+ # V17 is 15m-bar based; evaluating every 0.1s is wasteful.
+ eval_key = (symbol, strat.strategy_name)
+ if now < next_eval_time[eval_key]:
+     continue
+     next_eval_time[eval_key] = now + 5.0
+
+ strat_cooldown = strat.get_param(symbol, 'cooldown_seconds',
1800)
+ if now - strat.last_signal_time.get(symbol, 0) <
strat_cooldown:
+     continue
+
+ if "TRIGGERED" in strat.state.get(symbol, "") and \
+ now - strat.last_signal_time.get(symbol, 0) >=
strat_cooldown:
+     strat.state[symbol] = "SEARCHING"
+
+     strat.evaluate_macro_setup(symbol)
+     res, msg = strat.evaluate_trigger(symbol)
+     if res:
@@
- trade_tracker.process_signal(symbol, strat.strategy_name,
res, current_price)
+ accepted = trade_tracker.process_signal(symbol,
strat.strategy_name, res, current_price)
+ if accepted and res.get("direction", 0) != 0:
+     strat.last_signal_time[symbol] = time.time()
@@
- except Exception as e:
- except Exception as e:
+ try:
+     with open(os.path.join(os.path.dirname(os.path.abspath(__file__)),
'logs', 'strategy_eval_errors.log'), 'a', encoding='utf-8') as f:
+         f.write(f"[{datetime.now()}] {e}\n{traceback.format_exc()}\n")
+     except Exception:
+         pass
+     time.sleep(1.0)

```

8. Make Playwright page discovery non-mutating during live scraping

```

-def get_sorted_pages(context):
+def get_sorted_pages(context, allow_mutation=True):
@@
- if len(s9_pages) < len(PAGE_ASSETS):
+ if allow_mutation and len(s9_pages) < len(PAGE_ASSETS):
+     for p in pages:
@@
- if len(s9_pages) < len(PAGE_ASSETS):
+ if allow_mutation and len(s0_pages) < len(PAGE_ASSETS):
+     log_message(f"[Tabs] Found only {len(s9_pages)} layout tabs. Opening new layout/s9
tab(s)...")

```

In `CoinglassScraper.__init__`:

```

self.heal_attempts = {}
+ self._live_pages = []
+ self._live_pages_ts = 0.0

```

```

+ def _get_live_pages(self, force=False):
+     now = time.time()
+     stale = now - self._live_pages_ts > 30.0
+     closed = any(getattr(p, "is_closed", lambda: True)() for p in self._live_pages)
+     if force or stale or closed or not self._live_pages:
+         self._live_pages = get_sorted_pages(self.context, allow_mutation=False)
+         self._live_pages_ts = now
+     return self._live_pages

```

In `get_live_data()`:

```

- try:
-     pages = get_sorted_pages(self.context)
+ try:
+     pages = self._get_live_pages()
except Exception:
    pages = []

@@
- page.bring_to_front()
- time.sleep(0.5)
+ if page.is_closed():
+     continue
    page_results = page.evaluate(self.EXTRACTION_JS)

```

In the TV scraper thread, set timeouts:

```

thread_browser = pw.chromium.connect_over_cdp(CDP_BASE)
thread_context = thread_browser.contexts[0]
+ thread_context.set_default_timeout(5000)
+ thread_context.set_default_navigation_timeout(15000)

```

And avoid closing the setup CDP browser mid-engine:

```

- try:
-     print("\n[Setup] Disconnecting initial setup browser connection...")
-     browser.close()
- except Exception as cdp_disc_err:
-     print(f" [WARN] Setup browser disconnect error: {cdp_disc_err}")
+ print("\n[Setup] Keeping setup CDP connection open; Chrome lifecycle is managed at engine shutdown.")

```

For production-grade robustness, I'd still move Playwright scraping into a separate process with a heartbeat. A hung sync Playwright CDP call cannot be safely killed from another Python thread.

9. Remove hot-path debug file writes

```

+DEBUG_MAIN_LOOP = False
+
+def loop_debug(msg):
+     if not DEBUG_MAIN_LOOP:
+         return
+     try:
+         with open('main_loop_debug.log', 'a', encoding='utf-8') as dbg_file:
+             dbg_file.write(f"[{datetime.now()}] {msg}\n")
+     except Exception:
+         pass
+
@@
- with open('main_loop_debug.log', 'a') as dbg_file: dbg_file.write(f"[{datetime.now()}] Keyboard check done, starting UI update\n")
+ loop_debug("Keyboard check done, starting UI update")
@@
- with open('main_loop_debug.log', 'a') as dbg_file: dbg_file.write(f"[{datetime.now()}] Feed data fetched\n")
+ loop_debug("Feed data fetched")

```

Additional optimizations I would make next

- Filter Binance-supported assets once at startup.** Use `/fapi/v1/exchangeInfo` and build `BINANCE_SYMBOLS`. Only pass valid symbols to `FootprintScraper`, `OB` websocket, and raw kline strategies. Keep layout-only assets visible in dashboard but don't hammer Binance for invalid symbols.
- Reduce `EXTRACTION_JS` layout cost.**
 - Prefer `textContent` over `innerText` where possible.
 - Cache known legend elements per iframe after first successful scan.
 - Do not `bring_to_front()` in live scraping.
 - Scrape every 2-5s, not sub-second; most Coinglass TV indicators are not truly tick-level.
- Use JSONL for trade logs.**
 - Append `ENTRY/EXIT` events to `trade_events.jsonl`.
 - Periodically compact to summary JSON.
 - Avoid full JSON load/rewrite on every trade.
- Move Gemini meta-labeling off the strategy eval path.**
 - Cache verdicts by `(symbol, direction, regime_bucket)` for 60-300s.
 - Use deterministic local veto rules first.
 - Treat Gemini failures as configurable fail-open/fail-closed, not always `SAFE`.
- Fix V17 trailing semantics.**
 - Config says `trail_activation_r`, but execution compares it against ATR distance.
 - Use `risk_dist = abs(entry - initial_sl)` and activate at `trail_activation_r * risk_dist`.
- Add a watchdog for background threads.**
 - Heartbeat timestamp per thread.
 - If TV scraper heartbeat stale, restart Chrome/CDP scraper process.
 - If strategy eval heartbeat stale, log and restart/exit safely.

The most urgent fixes are: `TradeTracker` locking, locked feed-history snapshots, V17 config lookup, price-source freshness, cooldown enforcement, and Playwright live-scrape side-effect removal.

Your credit balance is negative - this run has stopped. Please top up to continue. [Upgrade](#)

AI Chat

The AI Costco — one membership →

Ask anything, create anything



GPT-5.5 Pro

